



CLAUDE CODE · FUNDAMENTOS · AULA 2

Contexto, Memória

e CLAUDE.md

Como o Claude aprende sobre o seu projeto e mantém contexto entre sessões — o contrato que transforma a qualidade das respostas.

CLAUDE.md

/context

escopo de sessão



O Claude não tem *memória nativa*

Cada sessão começa do zero — sem memória das conversas anteriores.

1 Sessão 1

- ✓ Python 3.11 · e-commerce
- ✓ PostgreSQL · Auth JWT
- ✓ Testes com pytest · deploy no Render

2 Sessão 2 — apagou tudo

? Qual linguagem · ? Qual projeto · ? Qual banco · ? Qual auth · ? Quais testes · ?
Onde faz deploy

✓ **A solução:** um arquivo `CLAUDE.md` na raiz do projeto (mais `/context`, `settings.json` e `/init`) dá ao Claude uma memória persistente que você controla.

O que é *contexto*?

Contexto = tudo que o Claude sabe **neste momento** sobre o seu projeto.



CLAUDE.md

Contexto **persistente**: stack, padrões, convenções e regras que você escreve.



Arquivos abertos

Contexto **ativo**: código, configs e docs que o Claude lê na sessão.



Histórico

Contexto de **sessão**: o diálogo atual — perguntas, respostas, decisões.



Settings

Contexto de **configuração**: ferramentas permitidas, env, permissões.

Só o CLAUDE.md e o settings.json sobrevivem ao fechar o terminal — o resto é da sessão

O *CLAUDE.md*

Um arquivo Markdown na raiz do projeto que o Claude lê automaticamente ao iniciar.

1º `./CLAUDE.md`

Raiz do projeto — lido primeiro

2º `~/.claude/CLAUDE.md`

Global do usuário

3º `./src/CLAUDE.md`

Subdiretório, se presente

```
# Projeto: E-commerce API
## Stack
- Python 3.11 + FastAPI
- PostgreSQL 15 + SQLAlchemy
- Redis para cache
## Padrões de código
- Sempre usar type hints
- Testes com pytest + fixtures
- Commits: feat/fix/chore
## Regras importantes
- NUNCA alterar models.py sem avisar
- Migrations: sempre via alembic
```

O que colocar no *CLAUDE.md*

01

Visão geral

Nome, objetivo, domínio de negócio e status atual do projeto.

02

Stack

Linguagem e versão, frameworks, banco e infraestrutura.

03

Padrões

Estilo de código, nomenclatura e estrutura de pastas.

04

Regras

O que NUNCA fazer, arquivos críticos e processos obrigatórios.

05

Comandos

Como rodar testes, build, deploy e comandos de banco.

06

Integrações

APIs externas, variáveis de ambiente e docs relevantes.

Gerenciando o *contexto ativo*

/context

Ver o contexto atual carregado

/context add <path>

Adicionar arquivo ao contexto

/context remove <path>

Remover arquivo do contexto

/context clear

Limpar todo o contexto de arquivos

Janela de contexto (~200K tokens)

 CLAUDE.md — 1.2 KB

 main.py — 3.4 KB

 models.py — 8.1 KB

 conversa atual — 2.3 KB

[disponível] — ~185 KB

O que *persiste*?

Nem tudo que o Claude aprendeu sobrevive ao fechar o terminal.

✓ Persiste

- ✓ CLAUDE.md (você escreveu)
- ✓ Arquivos modificados no disco
- ✓ Commits e histórico Git
- ✓ settings.json do projeto
- ✓ Checkpoints criados

✗ Não persiste

- ✗ Histórico de mensagens
- ✗ Arquivos no contexto ativo
- ✗ Decisões não documentadas
- ✗ Variáveis da sessão
- ✗ Estado da conversa



Como persistir

- ✓ Escrever no CLAUDE.md
- ✓ Salvar arquivos no disco
- ✓ Usar /checkpoint
- ✓ Documentar no código

Um CLAUDE.md que *funciona*

✓ Faça assim

- ✓ **Seja específico** — "Python 3.11 + FastAPI" > "uso Python"
- ✓ **Inclua exemplos** — o padrão de nomes (`get_user()`)
- ✓ **Liste regras críticas** — "use soft-delete, nunca DELETE"
- ✓ **Atualize sempre** — documentação viva
- ✓ **Inclua comandos** — rodar, testar, deploy

✗ Evite

- ✗ **Ser vago** — "sigo boas práticas" não diz nada
- ✗ **Arquivos gigantes** — > 200 linhas perde eficácia
- ✗ **Informação desatualizada** — confunde mais que ajuda
- ✗ **Misturar tudo** — separe projeto, padrões e regras
- ✗ **Colocar segredos** — NUNCA senhas ou tokens

Teste *rápido*

Ao fechar o terminal e abrir uma nova sessão, o que o Claude ainda saberá sobre o seu projeto?

A O histórico da conversa anterior

B O que está no CLAUDE.md e no settings.json

C Os arquivos que estavam no /context

D As decisões que você tomou verbalmente

Resposta: B. Só o que está **escrito no disco** persiste — CLAUDE.md, settings.json, arquivos, commits e checkpoints. Histórico de mensagens, contexto ativo e decisões não documentadas **desaparecem**. Por isso o CLAUDE.md é o "contrato" entre você e o Claude.



RESUMO DA AULA 2

O CLAUDE.md é o *contrato*

Quanto mais rico o CLAUDE.md, melhor o resultado. Ele é a memória persistente que você controla — stack, padrões, regras e comandos, num arquivo versionado com o projeto.

 CLAUDE.md persiste

 sessão não persiste

/context gerencia

Próxima aula → **Configuração de Projeto com /init.**